

CYBR 437: Secure Coding

Buffer Overflow-Towards Bypassing Defenses

Instructor: Abinash Borah

Outline

- Buffer Overflow Defenses in GCC and Linux
- Return Oriented Programming
- Function Pointers in C

GCC - Compile-Time Defenses

Stack Canary

- When a function starts, a secret value is placed onto the stack between local variables and the return address on the stack
- If a buffer overflow overwrites the return address, it corrupts the canary, and the program aborts
- Primary flags in GCC
 - **-fstack-protector**: Enables stack protection for functions that contain local character buffers larger than a specified size (8 bytes by default) or those that call `alloca()` - the buffer size threshold can be adjusted.
 - **-fstack-protector-strong**: Offers a better balance between security and performance by protecting more functions than `-fstack-protector` but not all of them, reducing performance overhead compared to the all option
 - **-fstack-protector-all**: Provides stack protection to all functions regardless of whether they appear vulnerable - strongest protection but can incur a performance penalty
 - **-fno-stack-protector**: Explicitly disables stack protection

GCC - Compile-Time Defenses (contd.)

Non-executable Stack

- Non-executable stack protection relies on CPU features (NX/XD bit) that prevent data in specific memory regions (like the stack) from being interpreted as executable code
- Non-executable stack is the default – GCC provides specific flags for explicit control

Primary flags:

- **-z noexecstack**: Explicitly tells the linker to mark the stack as non-executable
- **-z execstack**: Forces the stack to be marked as executable, even if object files request a non-executable stack

GCC - Compile-Time Defenses (contd.)

Position Independent Executable (PIE)

- Makes the program load at random memory locations each time it runs (works with ASLR)
- Modern Linux distributions often enable PIE by default in GCC
- GCC provides specific flags to control the generation and linking of PIE binaries

Primary flags:

- **-pie**: Tells the linker to produce a position-independent executable binary
- **-no-pie**: Explicitly disable the creation of a PIE binary, forcing a fixed-address executable

GCC - Compile-Time Defenses (contd.)

RELRO (RELocation Read-Only)

- RELRO makes some binary sections read-only to prevent Global Offset Table (GOT) overwrite
- GTO is a section inside programs (executables and shared libraries) that holds addresses of functions that are dynamically linked

There are two RELRO modes – partial and full

- Partial RELRO is the default setting in GCC and nearly all binaries have at least partial RELRO – enabled with `-Wl,-z,relro`
- Full RELRO makes the entire GOT read-only which removes the ability to perform a "GOT overwrite" attack (where the GOT address of a function is overwritten with the location of another function an attacker wants to run) – enabled with `-Wl,-z,relro,-z,now`

Linux OS/Kernel-level Defenses

ASLR (Address Space Layout Randomization)

- ASLR is the randomization of the place in memory where the stack, heap, and shared libraries reside – also the executable (if PIE enabled)
- Can be set with `echo X | sudo tee /proc/sys/kernel/randomize_va_space`
 - Where, X = 0/1/2 specifies Disabled/Partial/Full
- Can be checked with `cat /proc/sys/kernel/randomize_va_space`
 - Outputs 0/1/2 (as mentioned above)
- Full ASLR is the default in modern Linux versions

Linux OS/Kernel-level Defenses (contd.)

Data Execution Prevention (DEP)

- Marks certain memory regions, such as the stack and heap, as non-executable to prevent malicious code from running in data areas
- DEP is enabled by default in the Linux kernel on systems with compatible processors

Guard Pages

- Guard pages are inaccessible memory pages in Linux, placed at the end of a process's or thread's stack to detect stack overflows
- Any attempt by the stack to grow into a guard page triggers a segmentation fault (SIGSEGV) or an exception, thus terminating the offending process

Return Oriented Programming (ROP)

- Return-Oriented Programming is an exploitation technique used to execute code without injecting new shellcode
- Used especially when protections like non-executable stack are enabled
- Instead of putting malicious code on the stack, attackers reuse existing code already present in the program or libraries
- Attackers can gain control of the call stack to manipulate program control flow and execute chosen machine instruction sequences that are already present in memory – called “gadgets”
- ROP bypasses NX because, instead of executing injected code, it executes existing executable code in memory

What is a gadget?

- A gadget is a short sequence of existing (in memory) instructions that ends with a return instruction
- Examples:
 1.

```
pop rdi  
ret
```
 2.

```
mov rax, rdi  
ret
```
- These snippets already exist in memory - the attacker just redirects the control flow of a program to the gadgets

ROP Chain

- A ROP chain is a sequence of gadget addresses placed on the stack so that each “return” jumps to the next gadget, forming a “program”
- Something that looks like:

[padding to overflow buffer]

[address of gadget 1]

[value for gadget 1]

[address of gadget 2]

[value for gadget 2]

[address of gadget 3]

...

- Each gadget performs a small action and together they perform a complete attack

ROP Chain (contd.)

- Working of ROP chaining:
 1. Buffer overflow happens
 2. Attacker overwrites the saved return address
 3. Instead of returning to normal code, control returns to the first gadget
 4. Gadget runs and returns
 5. The return jumps to the next targeted gadget on the stack
 6. The chain continues...

Function Pointers in C

- A function pointer is a pointer that stores the address of a function – allowing functions to be passed as arguments and invoked dynamically
- It is useful in techniques like callback functions, event-driven programs, and polymorphism

- Example:

```
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int main() {

    // Declare a function pointer that matches the signature of add()
    int (*fptr)(int, int);

    // Assign the pointer to add()
    fptr = &add;

    // Call the function through the pointer
    printf("%d", fptr(10, 5));
    return 0;
}
```

Function Pointers in C (contd.)

- A useful application of function pointers is passing functions as arguments to other functions – this allows us to specify which function to call at runtime

- Example:

```
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}

void calculate(int a, int b, int (*op)(int, int)) {
    printf("%d\n", op(a, b));
}

int main() {
    // Passing different functions to calculate
    calculate(10, 20, add);
    calculate(25, 15, subtract);
    return 0;
}
```